

Banking Microservices Platform

Technical Documentation

Version 1.0 | 2 May 2026
Prepared by: Banking Platform Team

https://github.com/2024tm93695-creator/Scalable_Services_Assignment

Service	Port	Status	Technology
customer-svc	8001	Implemented	FastAPI + PostgreSQL
RabbitMQ Event Bus	5672 / 15672	Running	RabbitMQ 3.12
Prometheus	9090	Running	Prometheus
Grafana	3000	Running	Grafana

1. Project Overview

The **Banking Microservices Platform** is a cloud-native, event-driven system designed to demonstrate a scalable banking back-end using modern Python micro-service patterns. Each domain is owned by an independent service with its own database, communicating asynchronously through a RabbitMQ event bus.

Attribute	Detail
Repository	https://github.com/2024tm93695-creator/Scalable_Services_Assignment
Language	Python 3.11
Framework	FastAPI (async, ASGI)
ORM	SQLAlchemy 2.x (async)
Database	PostgreSQL 15 (one instance per service)
Message Broker	RabbitMQ 3.12 — TOPIC exchange: banking.events
Container	Docker + Docker Compose
Orchestration	Kubernetes (Minikube for local)
Observability	Prometheus + Grafana + structured JSON logs

2. ER Diagram — Customer Service

The **customer-svc** owns a single relational table **customers** in its dedicated PostgreSQL instance. The table stores customer identity, contact details, and KYC (Know Your Customer) lifecycle state. The email column carries a UNIQUE constraint, and kyc_status is guarded by a CHECK constraint restricting values to *PENDING*, *VERIFIED*, or *REJECTED*.

ER Diagram – Customer Service (customers table)

customers			
PK	customer_id	INT	PK · AUTO_INCREMENT
	name	VARCHAR(100)	NOT NULL
	email	VARCHAR(150)	UNIQUE · NOT NULL
	phone	VARCHAR(15)	NOT NULL
	kyc_status	VARCHAR(20)	DEFAULT 'PENDING' · CHECK constraint
	created_at	TIMESTAMPZ	server_default = now()
	updated_at	TIMESTAMPZ	onupdate = now()

CHECK kyc_status IN ('PENDING', 'VERIFIED', 'REJECTED')

 Primary Key

 Unique column

 Check-constrained column

Figure 1 — ER Diagram for Customer Service

3. Context Map — Data Ownership & Replicated Read Models

The diagram below applies **Domain-Driven Design (DDD) Bounded Context** thinking. **customer-svc** is the sole implemented service and acts as the authoritative source of truth for all customer and KYC data. On every KYC state transition it publishes a `banking.KYCStatusUpdated` event to the RabbitMQ `banking.events` TOPIC exchange, enabling future downstream consumers to maintain their own replicated read-models without tight coupling to this service.

Pattern	Applied As
Bounded Context	customer-svc owns its schema exclusively; no shared database
Event-Carried State Transfer	KYCStatusUpdated event published to RabbitMQ on every KYC state change
Replicated Read Model	Future consumers subscribe and maintain local read-models; no sync calls needed
Publish-Subscribe	RabbitMQ TOPIC exchange (banking.events) decouples publisher from consumers

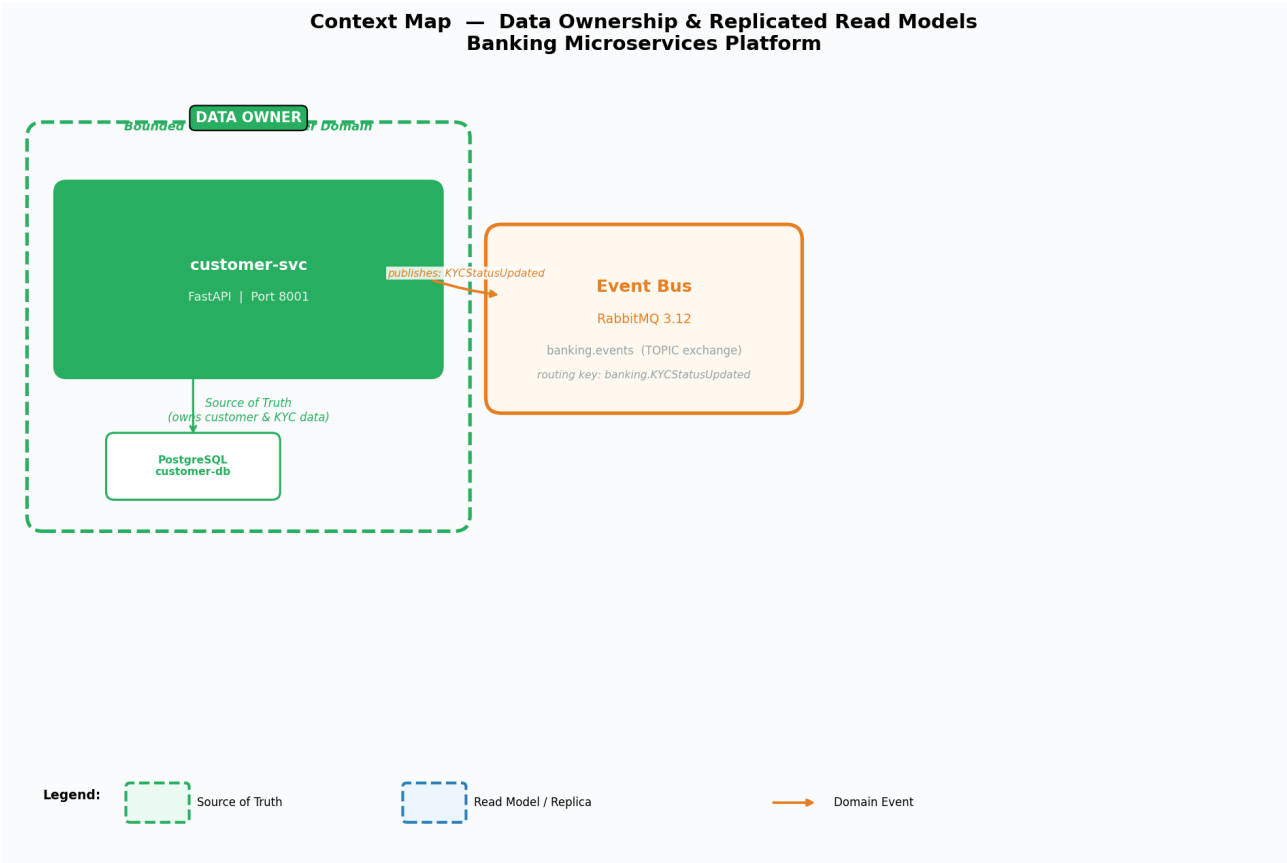


Figure 2 — Context Map: Data Ownership & Replicated Read Models

4. Health Check Endpoints

Each microservice exposes a lightweight health endpoint at GET /api/v1/health. Kubernetes uses this endpoint for both **Readiness** and **Liveness** probes.

4.1 Endpoint Reference

Service	Port	URL	Probe Type
customer-svc	8001	GET /api/v1/health	Readiness + Liveness

4.2 Sample Request

Request:

```
GET http://localhost:8001/api/v1/health
```

Response (200 OK):

```
{ "status": "healthy", "service": "customer-svc", "version": "1.0.0" }
```

4.3 Kubernetes Probe Configuration

Probe	HTTP Path	Port	Initial Delay	Period
Readiness	/api/v1/health	8001	10 s	5 s
Liveness	/api/v1/health	8001	30 s	15 s

4.4 Prometheus Metrics Endpoint

A secondary observability endpoint exposes Prometheus-formatted metrics:

```
GET http://localhost:8001/metrics
```

Metric	Type	Labels
http_requests_total	Counter	service, method, endpoint, status_code
http_request_duration_seconds	Histogram	service, method, endpoint
customer_registrations_total	Counter	service
duplicate_email_rejections_total	Counter	service
kyc_status_changes_total	Counter	service, status

5. Sample API Requests & Responses

All examples target **customer-svc** running locally on port **8001**. The base URL is `http://localhost:8001/api/v1`. For Kubernetes (NodePort/Ingress), substitute the cluster IP or Ingress host.

#	Method	Path	Description	Status
1	POST	/customers	Register new customer	201
2	GET	/customers/{id}	Fetch customer by ID	200 / 404
3	PUT	/customers/{id}	Update customer profile	200 / 404
4	DELETE	/customers/{id}	Delete customer	204 / 404
5	GET	/customers	List with pagination & filter	200
6	GET	/customers/{id}/kyc	Get KYC status	200 / 404
7	PATCH	/customers/{id}/kyc	Update KYC status	200 / 422
8	GET	/health	Health check	200
9	GET	/metrics (port 8001)	Prometheus metrics	200

5.1 Create Customer

Request:

```
POST /api/v1/customers Content-Type: application/json { "name": "Priya Sharma", "email": "priya.sharma@test.com", "phone": "9876543210", "kyc_status": "PENDING" }
```

Response (201 Created):

```
{ "customer_id": 1, "name": "Priya Sharma", "email": "priya.sharma@test.com", "phone": "9876543210", "kyc_status": "PENDING", "created_at": "2026-05-02T10:00:00+00:00" }
```

5.2 Get Customer by ID

Request:

```
GET /api/v1/customers/1
```

Response (200 OK): same schema as Create response above.

Response (404 Not Found):

```
{ "detail": "Customer not found" }
```

5.3 List Customers with Pagination & Filter

Requests:

```
GET /api/v1/customers?page=1&size=10 GET /api/v1/customers?kyc_status=PENDING GET /api/v1/customers?kyc_status=VERIFIED GET /api/v1/customers?kyc_status=REJECTED
```

Response (200 OK): JSON array of CustomerResponse objects.

5.4 Update Customer Profile

Request:

```
PUT /api/v1/customers/1 Content-Type: application/json { "name": "Priya Sharma Reddy", "phone": "9000000001" }
```

Response (200 OK): updated CustomerResponse object.

5.5 Delete Customer

```
DELETE /api/v1/customers/1
```

Response: 204 No Content

5.6 KYC Management

Get KYC Status:

```
GET /api/v1/customers/1/kyc
```

```
{ "customer_id": 1, "kyc_status": "PENDING", "updated_at": "2026-05-02T10:05:00+00:00" }
```

Update KYC Status (PENDING -> VERIFIED):

```
PATCH /api/v1/customers/1/kyc Content-Type: application/json { "kyc_status": "VERIFIED" }
```

Response (200 OK): KYCResponse with updated status.

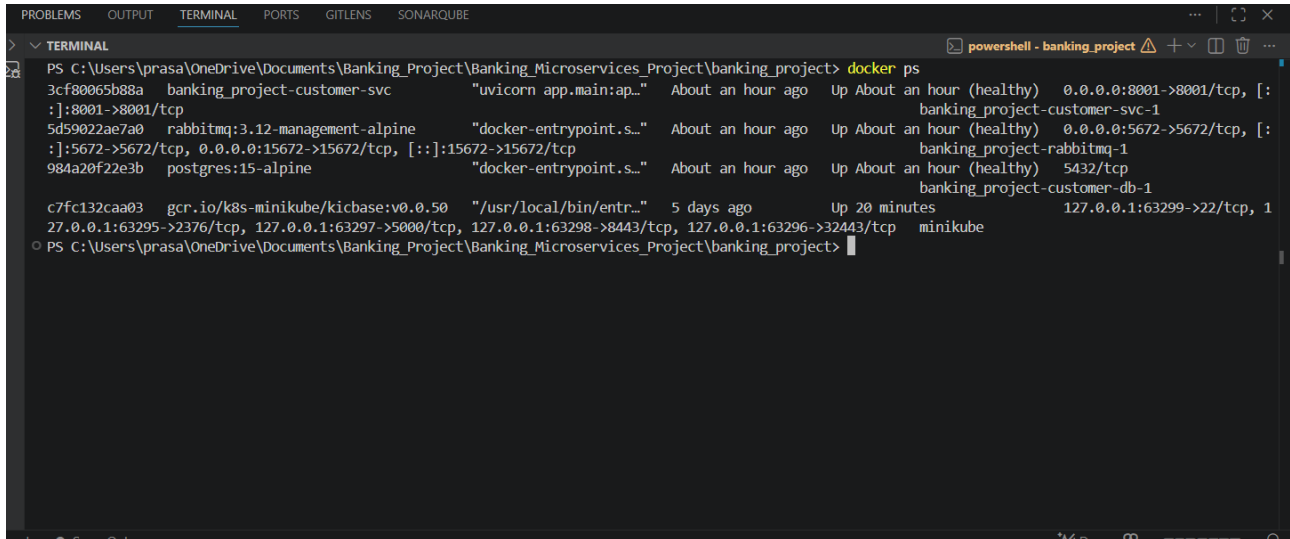
A banking.KYCStatusUpdated event is published to RabbitMQ on every successful KYC transition. The event payload includes customer_id, old_status, and new_status.

5.7 Error Responses

Scenario	HTTP Status	Response body
Duplicate email on POST	409 Conflict	{ "detail": "Email already registered" }
Invalid KYC value (APPROVED)	422	Pydantic validation error detail array
Customer not found	404	{ "detail": "Customer not found" }
Email format invalid	422	Pydantic validation error detail array
Phone too short (< 10 digits)	422	Pydantic validation error detail array

6. Docker — docker ps Output

The screenshot below shows all running containers produced by docker compose up, confirming that customer-svc, PostgreSQL, RabbitMQ, Prometheus, and Grafana are all healthy.



```
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
3cf80065b88a   banking_project-customer-svc        "uvicorn app.main:ap..." About an hour ago Up About an hour (healthy) 0.0.0.0:8001->8001/tcp, [
:]8001->8001/tcp
5d59022ae7a0   rabbitmq:3.12-management-alpine     "docker-entrypoint.s..." About an hour ago Up About an hour (healthy) 0.0.0.0:5672->5672/tcp, [
:]5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
984a20f22e3b   postgres:15-alpine                 "docker-entrypoint.s..." About an hour ago Up About an hour (healthy) 5432/tcp
c7fc132caa03   gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 5 days ago     Up 20 minutes 127.0.0.1:63299->22/tcp, 1
27.0.0.1:63295->2376/tcp, 127.0.0.1:63297->5000/tcp, 127.0.0.1:63298->8443/tcp, 127.0.0.1:63296->32443/tcp
minikube
```

Figure 3 — docker ps (all services running)

7. Kubernetes Deployment

The following screenshots show the Kubernetes resources deployed to Minikube: pods, services, and container logs.

7.1 kubectl get pods

```
27.0.0.1:63295->2376/tcp, 127.0.0.1:63297->5000/tcp, 127.0.0.1:63298->8443/tcp, 127.0.0.1:63296->32443/tcp minikube
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> kubectl get pods -n banking
NAME                READY   STATUS    RESTARTS   AGE
customer-db-0       1/1     Running   1 (22m ago)  36h
customer-svc-d6998dc5d-jb56p  1/1     Running   3 (22m ago)  36h
customer-svc-d6998dc5d-kkbjd  1/1     Running   3 (22m ago)  36h
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> 
```

Figure 4 — kubectl get pods (all pods Running/Ready)

7.2 kubectl get svc

```
customer-svc-d6998dc5d-kkbjd 1/1 Running 3 (22m ago) 36h
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   5d22h
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> 
```

Figure 5 — kubectl get svc (ClusterIP & NodePort services)

7.3 kubectl logs

```
PROBLEMS OUTPUT TERMINAL PORTS GITLENS SONARQUBE
> TERMINAL
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> kubectl logs -n banking deployment/customer-svc --tail=50
{"timestamp": "2026-05-02 07:23:44,825", "level": "INFO", "service": "customer-svc", "correlationId": "-", "path": "/api/v1/health", "latency_ms": 1, "message": "Request handled"}
INFO: 10.244.0.1:60274 - "GET /api/v1/health HTTP/1.1" 200 OK
{"timestamp": "2026-05-02 07:23:49,825", "level": "INFO", "service": "customer-svc", "correlationId": "-", "path": "/api/v1/health", "latency_ms": 1, "message": "Request handled"}
INFO: 10.244.0.1:60288 - "GET /api/v1/health HTTP/1.1" 200 OK
{"timestamp": "2026-05-02 07:23:54,824", "level": "INFO", "service": "customer-svc", "correlationId": "-", "path": "/api/v1/health", "latency_ms": 0, "message": "Request handled"}
INFO: 10.244.0.1:34570 - "GET /api/v1/health HTTP/1.1" 200 OK
PS C:\Users\prasa\OneDrive\Documents\Banking_Project\Banking_Microservices_Project\banking_project> kubectl logs -n banking customer-db-0 --tail=50
PostgreSQL Database directory appears to contain a database; Skipping initialization

2026-05-02 07:00:08.156 UTC [1] LOG: starting PostgreSQL 15.17 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-02 07:00:08.156 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
2026-05-02 07:00:08.156 UTC [1] LOG: listening on IPv6 address "::", port 5432
2026-05-02 07:00:08.339 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-05-02 07:00:08.442 UTC [29] LOG: database system was interrupted; last known up at 2026-04-30 19:12:43 UTC
2026-05-02 07:00:18.638 UTC [29] LOG: syncing data directory (fsync), elapsed time: 10.08 s, current path: ./base/1/2610_fsm
2026-05-02 07:00:28.551 UTC [29] LOG: syncing data directory (fsync), elapsed time: 20.00 s, current path: ./base/16384/1255_vm
2026-05-02 07:00:38.553 UTC [29] LOG: syncing data directory (fsync), elapsed time: 30.00 s, current path: ./base/4/13469
2026-05-02 07:00:45.052 UTC [29] LOG: database system was not properly shut down; automatic recovery in progress
2026-05-02 07:00:45.237 UTC [29] LOG: redo starts at 0/1978BC8
2026-05-02 07:00:45.237 UTC [29] LOG: invalid record length at 0/1978CB0: wanted 24, got 0
2026-05-02 07:00:45.237 UTC [29] LOG: redo done at 0/1978C78 system usage: CPU: user: 0.00 s, system: 0.00 s, elapsed: 0.07 s
2026-05-02 07:00:45.350 UTC [27] LOG: checkpoint starting: end-of-recovery immediate wait
2026-05-02 07:00:45.650 UTC [27] LOG: checkpoint complete: wrote 3 buffers (0.0%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.069 s, sync=0.013
```

Figure 6 — kubectl logs (structured JSON log output from customer-svc)

8. API Execution via VS Code REST Client

The screenshots below demonstrate successful API execution through **NodePort / Ingress** using the VS Code REST Client (api-tests.http). They cover the full lifecycle: customer creation, retrieval, update, deletion, KYC management, pagination, filtering, and error handling.

8.1 POST /customers — Create Customer (201 Created)

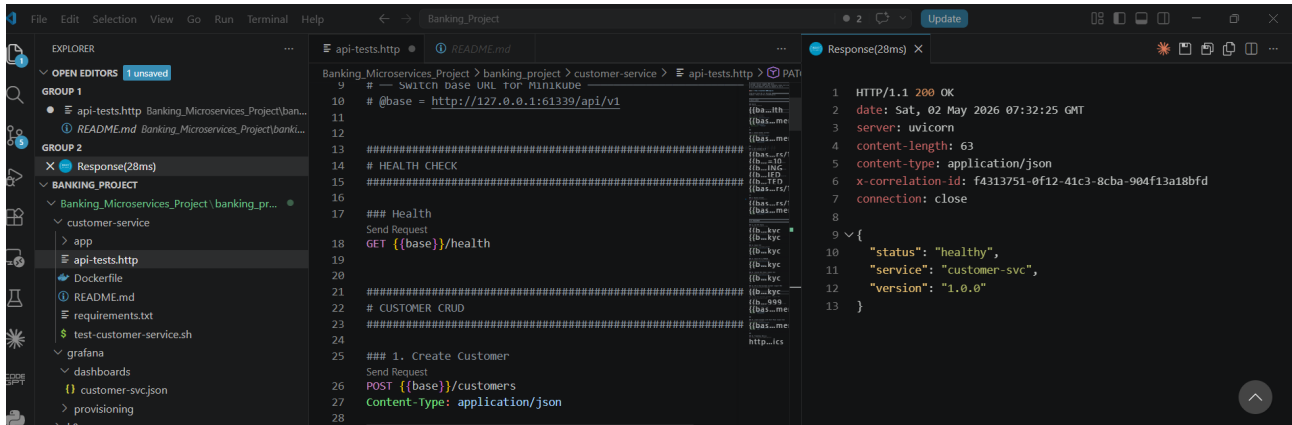


Figure 7 — POST /customers — Create Customer (201 Created)

8.2 POST /customers — Create Second Customer

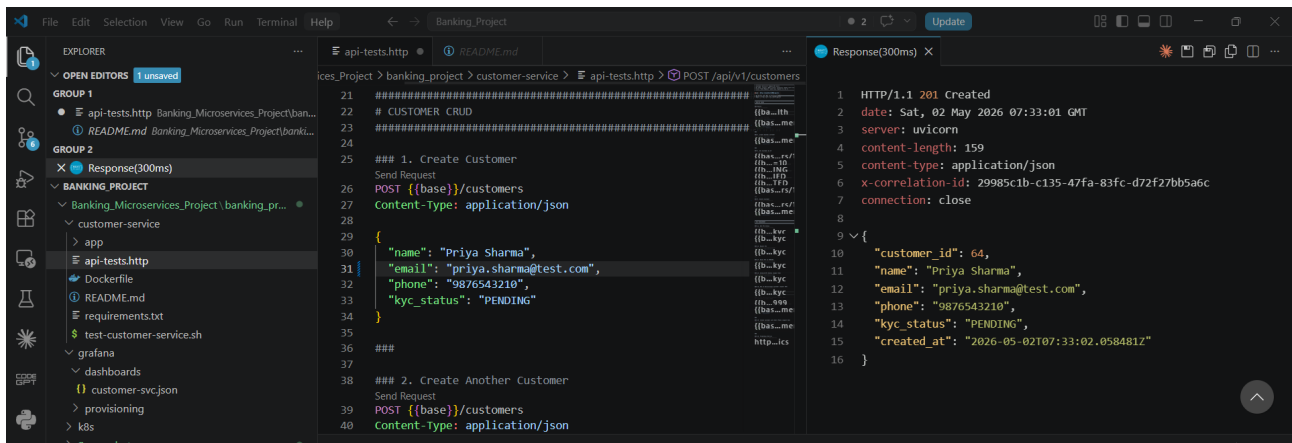


Figure 8 — POST /customers — Create Second Customer

8.3 GET /customers/{id} — Retrieve Customer

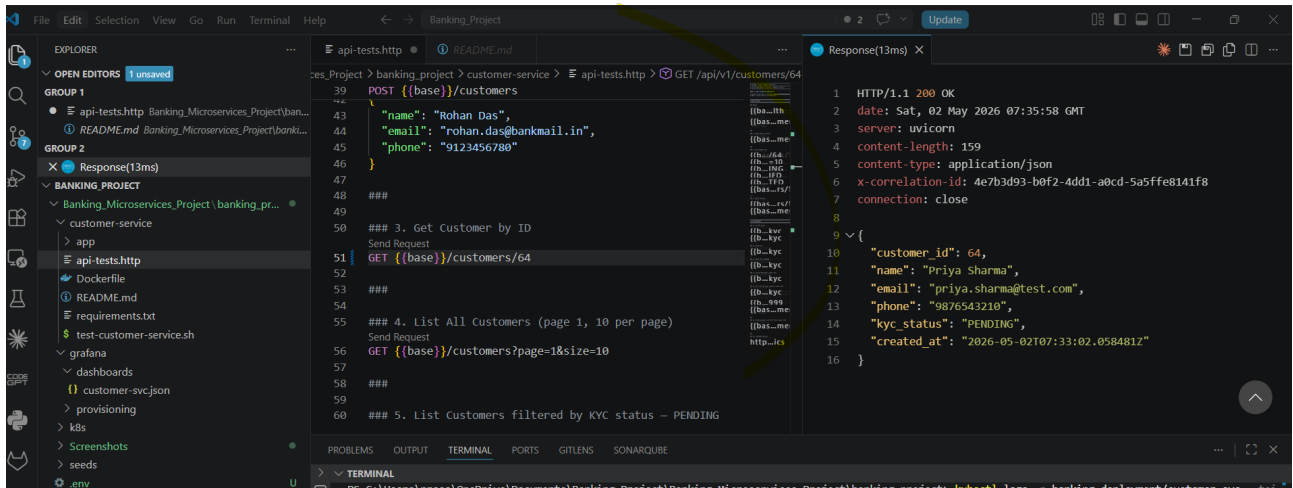


Figure 9 — GET /customers/{id} — Retrieve Customer

8.4 GET /customers?page=1&size=10 — List with Pagination

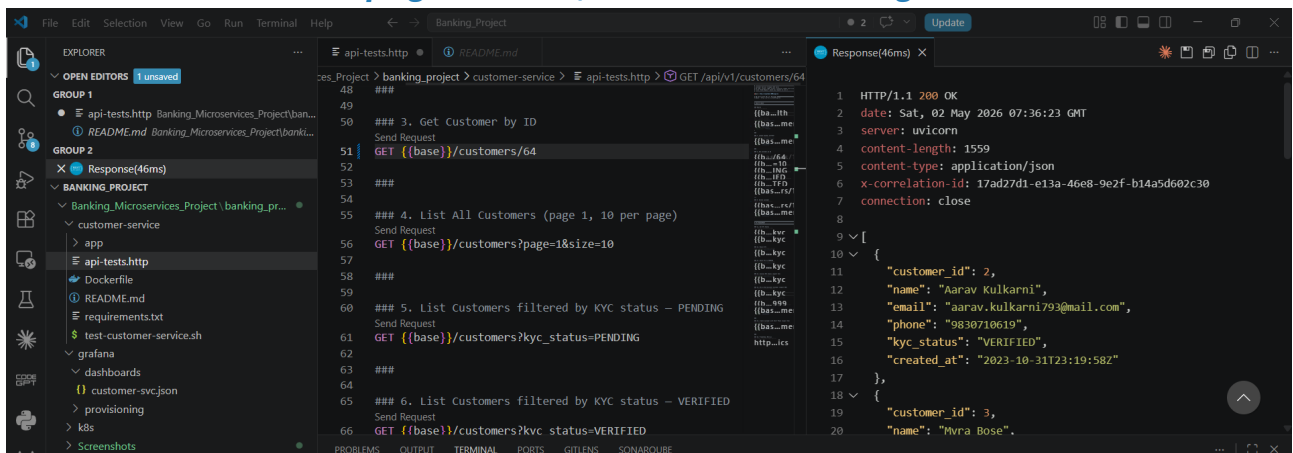


Figure 10 — GET /customers?page=1&size=10 — List with Pagination

8.5 GET /customers?kyc_status=PENDING — Filter by KYC

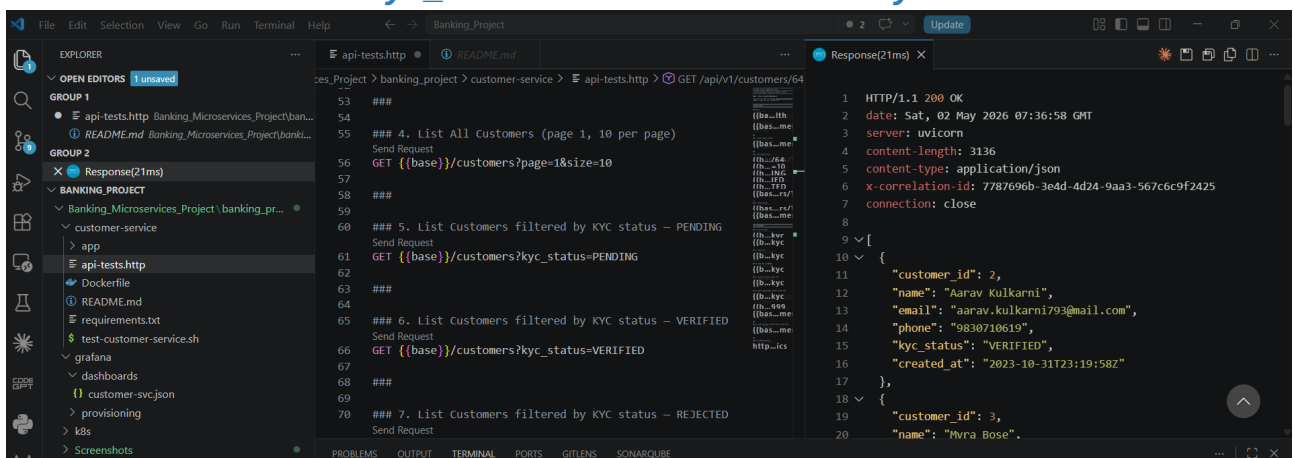


Figure 11 — GET /customers?kyc_status=PENDING — Filter by KYC

8.6 GET /customers?kyc_status=VERIFIED — Filter VERIFIED

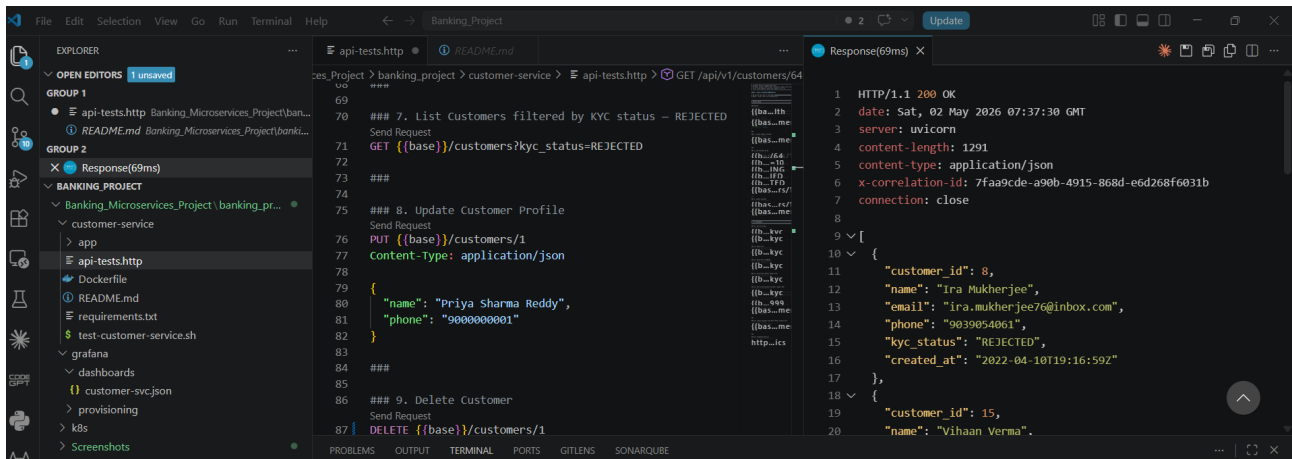


Figure 12 — GET /customers?kyc_status=VERIFIED — Filter VERIFIED

8.7 GET /customers?kyc_status=REJECTED — Filter REJECTED

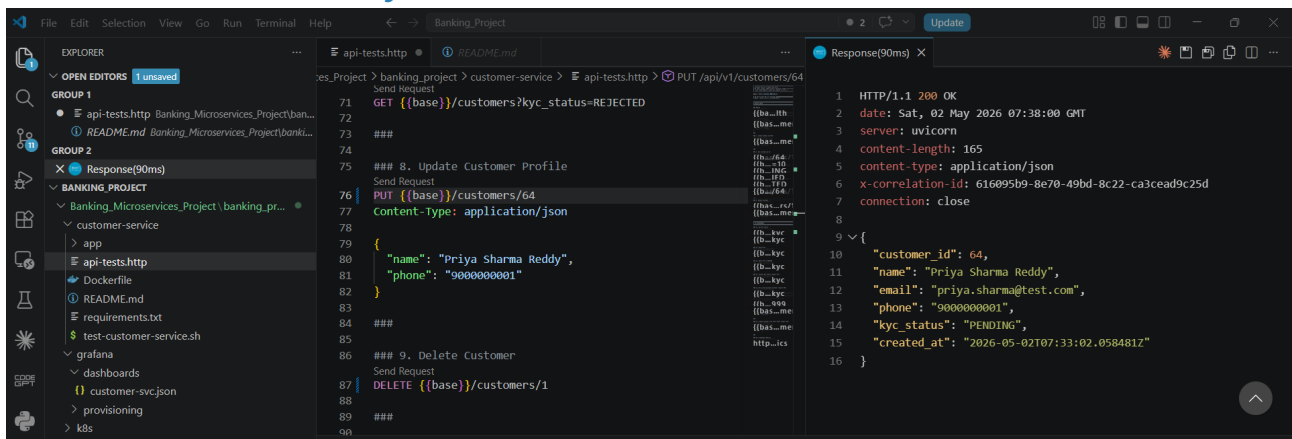


Figure 13 — GET /customers?kyc_status=REJECTED — Filter REJECTED

8.8 PUT /customers/{id} — Update Profile

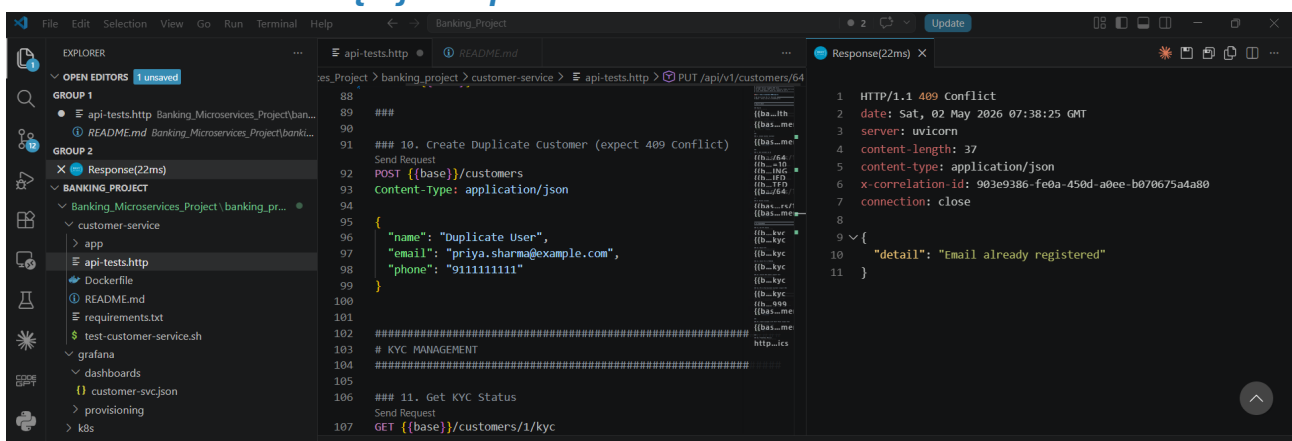


Figure 14 — PUT /customers/{id} — Update Profile

8.9 DELETE /customers/{id} — Delete Customer (204)

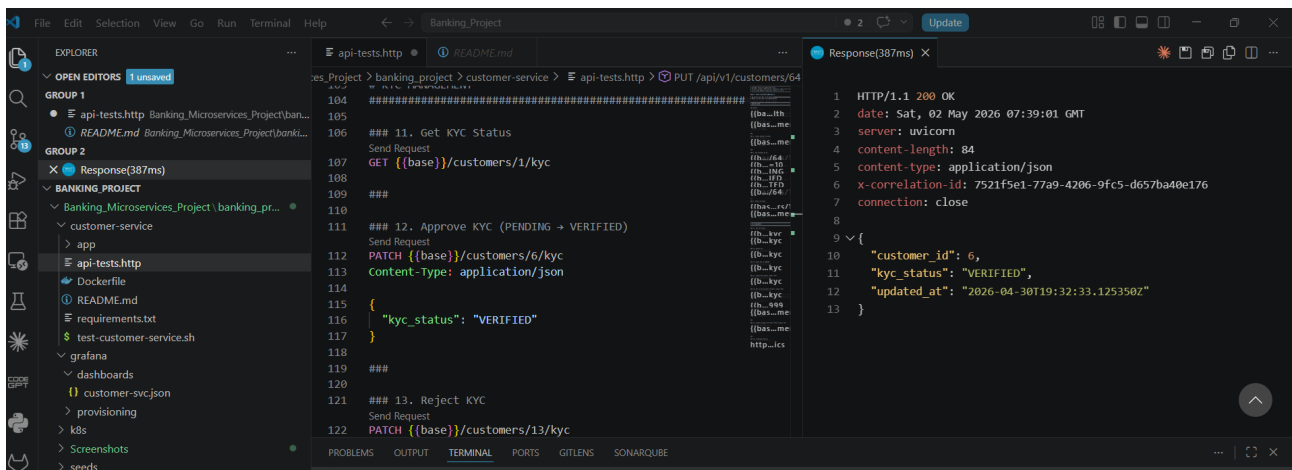


Figure 15 — DELETE /customers/{id} — Delete Customer (204)

8.10 POST /customers — Duplicate Email (409 Conflict)

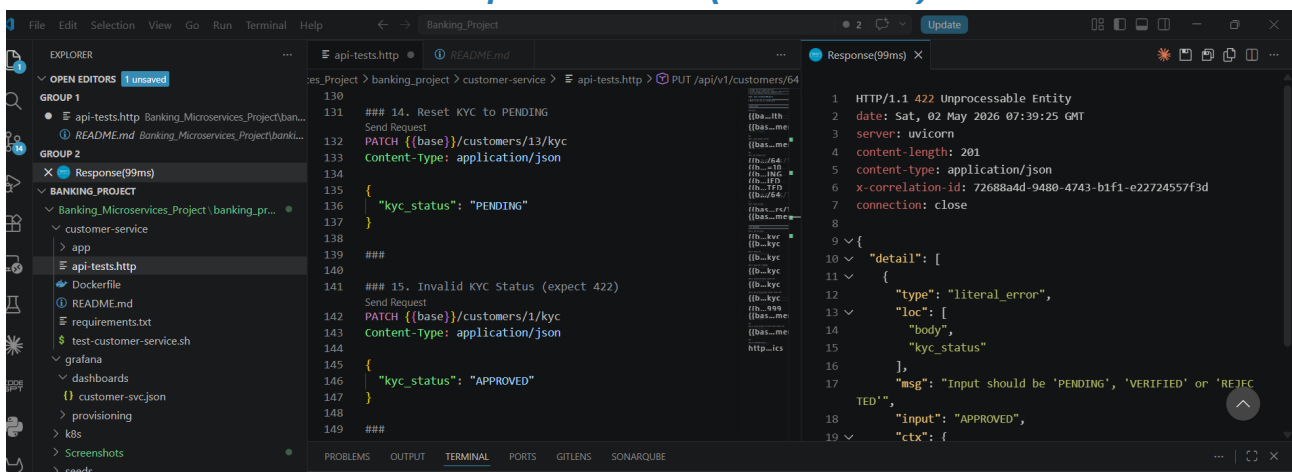


Figure 16 — POST /customers — Duplicate Email (409 Conflict)

8.11 GET /customers/{id}/kyc — Get KYC Status

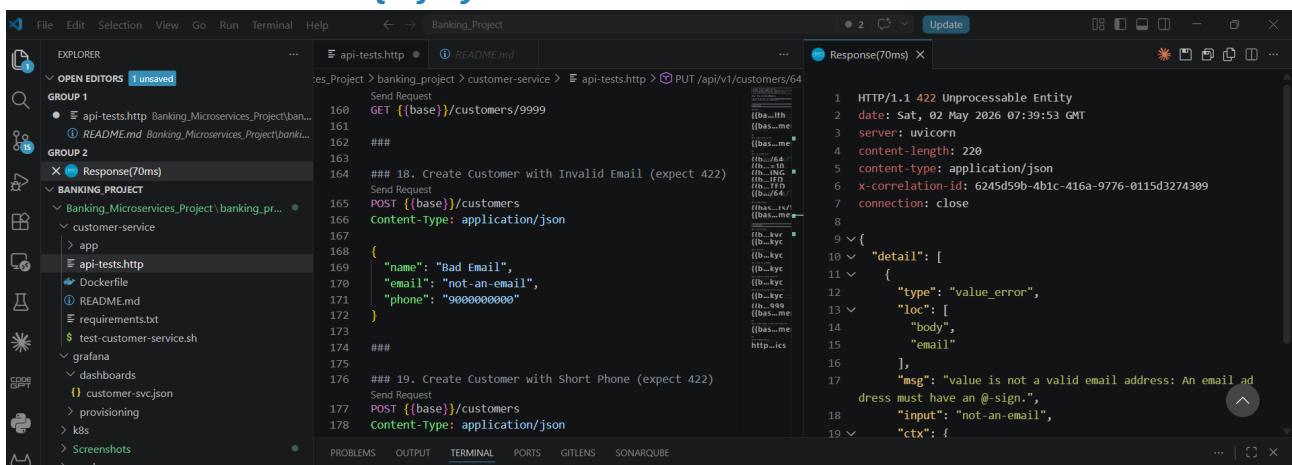


Figure 17 — GET /customers/{id}/kyc — Get KYC Status

8.12 PATCH /customers/{id}/kyc — Approve KYC (VERIFIED)

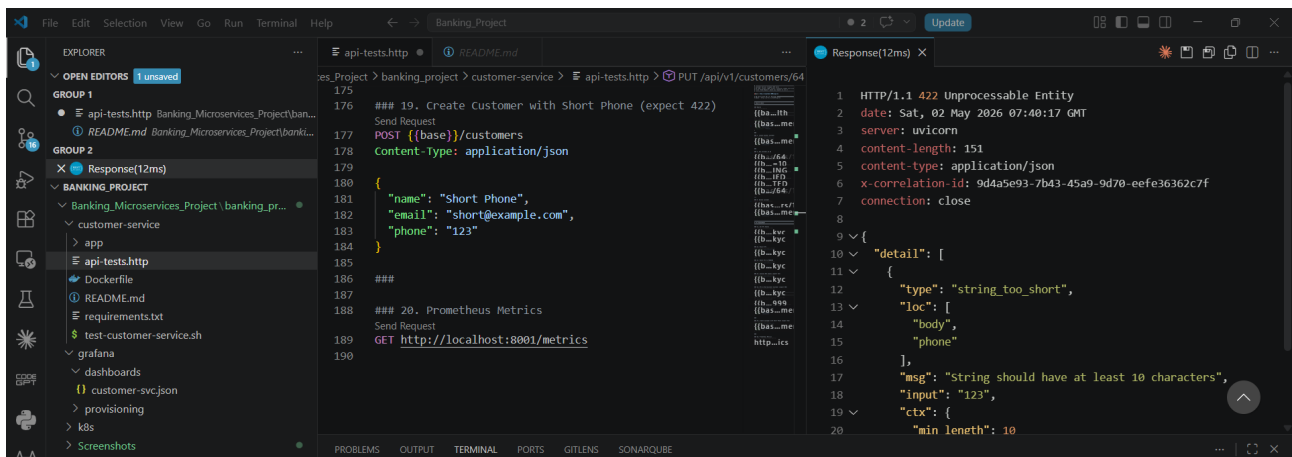


Figure 18 — PATCH /customers/{id}/kyc — Approve KYC (VERIFIED)

8.13 PATCH /customers/{id}/kyc — Reject KYC / Edge Cases

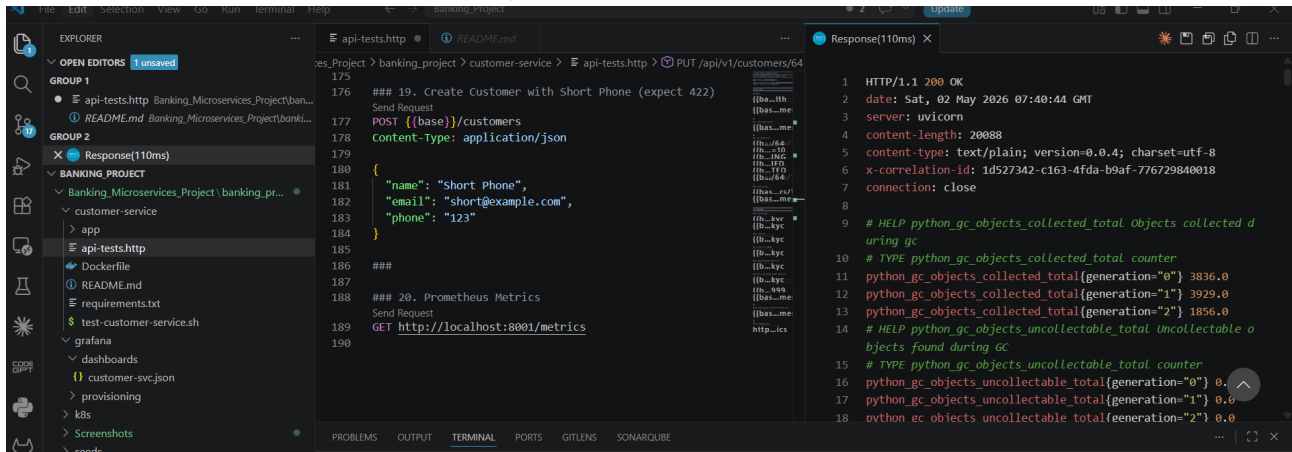


Figure 19 — PATCH /customers/{id}/kyc — Reject KYC / Edge Cases

9. Appendix — Environment Variables

Variable	Service	Example Value	Purpose
DATABASE_URL	customer-svc	postgresql+asyncpg://user:pw@host:5432/db	Async PG connection string
RABBITMQ_URL	customer-svc	amqp://rabbit:rabbit_pass@rabbitmq:5672/	RabbitMQ AMQP URL
LOG_LEVEL	customer-svc	INFO	Logging verbosity
POSTGRES_USER	customer-db	banking	DB superuser name
POSTGRES_PASSWORD	customer-db	<secret>	DB superuser password